

Universidade Federal de São Paulo (UNIFESP)
Instituto de Ciência e Tecnologia (ICT)

Cálculo Numérico
Prof. Luiz Leduino de Salles Neto

Trabalho Computacional 2

Sistemas lineares e sistemas não-lineares (partes 1 e 2)

Integrantes

Gabriel Lemos da Silva	RA: 140971
Davi Giuberti de Souza	RA: 168631
Rodrigo Ezequiel Silva dos Anjos	RA: 188215
Mayara Martinez Boaro	RA: 188197

Repositório com o código: <https://github.com/DaviGiuberti/TC2>

São José dos Campos
2 de outubro de 2026

Sumário

1	Introdução	2
2	Métodos e código usados	2
2.1	Sistemas lineares	2
2.2	Método de Newton	2
2.3	Ordem de convergência	3
3	Parte 1	3
3.1	Exercício 1: custo anual por aluno	3
3.2	Exercício 2: rede de resistores	5
3.3	Exercício 3: parábola e elipse	8
3.4	Exercício 4	9
4	Parte 2: rede de adução por gravidade	10
4.1	Item (a): coeficientes e Jacobiana	10
4.2	Item (b): Newton a partir de $x^{(0)} = (0,10; 0,10; -0,05; 70)$	11
4.3	Item (c): o ponto $x^{(0)} = (0; 0; 0; 70)$	12
4.4	Item (d): sentido do escoamento e velocidades	13
4.5	Item (e): demanda crítica	14
4.6	Item (f): sensibilidade ao envelhecimento da adutora 1	15
4.7	Conferência com <code>fsolve</code>	15
5	Código-fonte	16
6	Conclusão	16

1 Introdução

Neste trabalho resolvemos os quatro exercícios da parte 1 do TC2 (dois sistemas lineares e dois sistemas não-lineares) e o problema da rede de adução por gravidade da parte 2. Usamos Python 3 com `numpy`, e as figuras foram feitas com `matplotlib`. O `scipy` só aparece no fim da parte 2, para conferir as respostas do Newton por outro caminho.

Cada exercício tem um script próprio, que grava na pasta `resultados/` as tabelas e figuras usadas aqui. Os números citados no texto também saem dos scripts, gravados num arquivo de macros do `LATEX`. Assim nenhum resultado foi digitado à mão. O código completo está no repositório (Seção 5).

Seguimos a notação das aulas: $x^{(k)}$ é o iterado k , $s^{(k)}$ é o passo de Newton e J é a matriz Jacobiana. Todas as normas são a norma do máximo, $\|v\|_\infty = \max_i |v_i|$ (slides de Normas Vetoriais, slide 7). Escolhemos essa norma porque ela garante que nenhuma componente erra mais do que o valor calculado, e é esse tipo de garantia que os exercícios pedem.

2 Métodos e código usados

2.1 Sistemas lineares

Para o método direto usamos a eliminação de Gauss com pivotamento parcial, seguindo o pseudocódigo dos slides 14 e 18 da aula de Sistemas Lineares, e a retro-substituição do slide 7. A única mudança em relação ao pseudocódigo é que o laço mais interno (em j) virou uma operação com fatias do `numpy`.

Os métodos iterativos são o Gauss-Jacobi (slide 23) e o Gauss-Seidel (slide 27), escritos do mesmo jeito que nos slides. Nos dois o laço para quando o erro relativo

$$d^{(k)} = \frac{\|x^{(k)} - x^{(k-1)}\|_\infty}{\|x^{(k)}\|_\infty}$$

fica menor que a tolerância ε (slide 24) ou quando chega no número máximo de iterações. O critério das linhas e o de Sassenfeld (slides 30 e 31) também viraram funções. Está tudo em `src/metodos.py`.

2.2 Método de Newton

O ponto de partida foi a função `newton(F, JF, x0, TOL, N)` do livro da UFRGS que veio no enunciado da parte 1. Mantivemos a assinatura e a estrutura do laço, mas fizemos três mudanças.

A primeira é a mais importante. O código original calcula o passo usando a inversa da Jacobiana. O slide 13 da aula diz para nunca calcular J^{-1} , porque inverter custa cerca de três vezes mais e amplifica os erros de arredondamento. Por isso trocamos essa linha por uma chamada que resolve o sistema linear $J(x^{(k)}) s^{(k)} = -F(x^{(k)})$ do slide 9:

```
# livro da UFRGS
delta = -np.linalg.inv(JF(x)).dot(F(x))
# nossa versao (slide 13: resolve o sistema, nao inverte)
s = np.linalg.solve(JF(x), -Fx)
```

O `np.linalg.solve` resolve por fatoração LU com pivotamento parcial (rotina `gesv` do LAPACK), que é o que o passo 3 do algoritmo do slide 12 pede.

A segunda mudança foi guardar, em cada iteração, $x^{(k)}$, $\|F(x^{(k)})\|_\infty$ e $\|s^{(k)}\|_\infty$. Todas as tabelas de Newton deste relatório saem desse histórico.

A terceira foi nos critérios de parada. O código da UFRGS só olha o tamanho do passo, $\|s^{(k)}\|_\infty < \text{TOL}$, que é o critério do slide 12. Acrescentamos os dois critérios do slide 25: o resíduo $\|F(x^{(k)})\|_\infty < \varepsilon_1$ e o número máximo de iterações. O laço para no primeiro que acontecer e a função devolve qual foi, como o slide 24 recomenda. Quando estoura o número máximo de iterações o original levanta um `NameError`. Nós preferimos devolver o último iterado com um aviso, para o script conseguir montar a tabela mesmo assim. Se a Jacobiana for singular, o `np.linalg.solve` levanta um `LinAlgError` e quem chamou decide o que fazer. Isso aparece no item (c) da parte 2.

2.3 Ordem de convergência

Para ver a ordem de convergência fizemos como nos slides 16 e 17. Calculamos $e_k = \|x^{(k)} - x^*\|_\infty$ e olhamos o expoente de e_k . Nas tabelas, a coluna “exp.” é $\lfloor \log_{10} e_k \rfloor$, que é o número que o slide 17 acompanha ($-2 \rightarrow -3 \rightarrow -6 \rightarrow -13$ no exemplo da aula). Como não conhecemos x^* exato, usamos como referência o próprio Newton continuado até o passo ficar no nível do epsilon de máquina ($\|s\|_\infty \leq 4 \epsilon_{\text{máq}} \|x\|_\infty$). Também estimamos a ordem por

$$p_k \approx \frac{\log(e_k/e_{k-1})}{\log(e_{k-1}/e_{k-2})},$$

que deve dar perto de 2 quando a convergência é quadrática. Essa conta só tem sentido quando o erro está caindo e ainda está acima do arredondamento, então só mostramos p_k nesses casos.

3 Parte 1

3.1 Exercício 1: custo anual por aluno

Modelagem. A parte mais difícil deste exercício foi entender o enunciado. Percebemos que as colunas da tabela somam 100 % e as linhas não. Então cada coluna descreve de onde vêm os alunos matriculados nos cursos de um college. A frase “the enrollment consists of 70% from Sciences, 20% from Engineering, and 10% from Computer Science” é exatamente a primeira coluna, o que confirma essa leitura. Com isso, os valores 4000, 1000 e 2000 são as matrículas nos cursos de cada college, e não o número de alunos que pertencem a cada um.

Chamamos de x_i o custo anual de educar um aluno do college i ($i = 1$ Sciences, $i = 2$ Engineering, $i = 3$ Computer Science), de N_j as matrículas nos cursos do college j , de B_j o orçamento do college j e de p_{ij} a porcentagem da tabela na linha i e coluna j . A

hipótese do modelo é que o orçamento de cada college paga todos os alunos que ele atende, cada um pelo custo do seu college de origem:

$$\sum_{i=1}^3 p_{ij} N_j x_i = B_j, \quad j = 1, 2, 3.$$

Escrevendo uma equação para cada college, ficamos com o sistema $Ax = b$

$$\begin{pmatrix} 2.800 & 800 & 400 \\ 100 & 900 & 0 \\ 300 & 200 & 1.500 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 16.000.000 \\ 5.000.000 \\ 8.000.000 \end{pmatrix}.$$

Chegamos a pensar em só dividir cada orçamento pelas matrículas, mas isso dá um custo por vaga que não usa a tabela de distribuição. Por isso descartamos essa leitura.

Método. Resolvemos com a eliminação de Gauss com pivotamento parcial (slides 14 e 18) e conferimos com `np.linalg.solve`. Em cada etapa o maior elemento da coluna já estava na posição do pivô, então o pivotamento não precisou trocar linhas (o contador de trocas do script terminou em 0). Como o slide 13 recomenda, verificamos a solução calculando o resíduo.

Resultados. A Tabela 1 mostra a solução. O resíduo ficou $\|b - Ax\|_\infty = 1,9 \times 10^{-9}$. Parece grande, mas os orçamentos são da casa dos milhões de dólares. Dividindo por $\|b\|_\infty$ o resíduo relativo é $1,2 \times 10^{-16}$, que é o nível do arredondamento. A diferença entre a nossa eliminação e o `numpy` foi de $9,1 \times 10^{-13}$.

Tabela 1: Exercício 1: custo anual de um aluno de cada college.

College	x_i (Gauss)	x_i (<code>np.linalg.solve</code>)	Custo (US\$)
Sciences	3.685,393258	3.685,393258	3.685,39
Engineering	5.146,067416	5.146,067416	5.146,07
Computer Science	3.910,112360	3.910,112360	3.910,11

Também fizemos uma conferência de consistência. Para cada college de origem, somamos os alunos dele que aparecem nos cursos dos três colleges e obtivemos 3.200, 1.900 e 1.900 alunos. Multiplicando pelos custos encontrados, o total dá US\$ 29.000.000,00, que é a soma dos três orçamentos. Então o modelo distribui o dinheiro todo, sem sobra e sem falta.

Resposta. Arredondando ao centavo, o custo anual de um aluno é de US\$ 3.685,39 em Sciences, US\$ 5.146,07 em Engineering e US\$ 3.910,11 em Computer Science. Engineering sai mais caro. Isso combina com os dados, porque é o college com maior orçamento por matrícula e quase todos os alunos dos seus cursos são do próprio college.

3.2 Exercício 2: rede de resistores

Sistema. Com os valores do enunciado, o sistema obtido pelas leis de Ohm e de Kirchhoff fica

$$\begin{pmatrix} 350 & -100 & 0 & -150 \\ -100 & 350 & -150 & 0 \\ 0 & -150 & 550 & -200 \\ -150 & 0 & -200 & 550 \end{pmatrix} \begin{pmatrix} i_1 \\ i_2 \\ i_3 \\ i_4 \end{pmatrix} = \begin{pmatrix} -10 \\ 12 \\ 0 \\ 0 \end{pmatrix},$$

com resistências em ohms, tensões em volts e correntes em ampères.

Dominância diagonal. O enunciado diz que a matriz é diagonalmente dominante e conferimos isso numericamente (Tabela 2). Em todas as linhas o elemento da diagonal é maior, em módulo, que a soma dos outros. Portanto a matriz é estritamente diagonal dominante. O maior α_i é $\alpha = 0,7143 < 1$, e pelo critério das linhas (slide 30) tanto Jacobi quanto Gauss-Seidel convergem para qualquer $x^{(0)}$. O critério de Sassenfeld (slide 31) deu $\beta = 0,7143$. Aqui β saiu igual a α porque o maior valor está na primeira linha, e lá $\beta_1 = \alpha_1$ sempre. Nas outras linhas β_i ficou menor que α_i , como o slide 31 diz que acontece.

Tabela 2: Exercício 2: critério das linhas e critério de Sassenfeld.

Linha i	$ a_{ii} $	$\sum_{j \neq i} a_{ij} $	α_i	β_i
1	350	250	0,7143	0,7143
2	350	250	0,7143	0,6327
3	550	350	0,6364	0,5362
4	550	350	0,6364	0,3898

Para ter uma ideia da velocidade calculamos também o raio espectral das matrizes de iteração dos dois métodos (slides 21 e 27). Deu $\rho(C_J) = 0,6688$ e $\rho(C_{GS}) = 0,4635$. Como o erro é multiplicado mais ou menos por ρ a cada iteração, já esperávamos que Seidel fosse bem mais rápido.

Métodos iterativos. Rodamos os dois métodos a partir de $x^{(0)} = 0$, com $\varepsilon = 10^{-6}$ no erro relativo e no máximo 200 iterações. Jacobi parou pela tolerância depois de 34 iterações e Seidel depois de 17. Seidel precisou de metade das iterações, que é o que o slide 26 comenta. As Tabelas 3 e 4 têm todas as iterações, e a Figura 1 mostra o erro em relação à solução direta. Em escala logarítmica as duas curvas são praticamente retas, que é o sinal de convergência linear que aparece no slide 29, e a de Seidel é mais inclinada.

Comparação com o método direto. Resolvemos o mesmo sistema com Gauss com pivotamento parcial. De novo o contador de trocas de linha terminou em 0, e o resultado bateu com o `np.linalg.solve` (a maior diferença entre os dois foi 0). A Tabela 5 compara os três resultados. No final, o erro de Jacobi em relação à solução direta foi de $1,4 \times 10^{-8}$ e o de Seidel foi de $1,7 \times 10^{-8}$. O slide 24 avisa que variação pequena entre iterações não garante erro pequeno, por isso conferimos também o resíduo, que ficou pequeno nos dois casos.

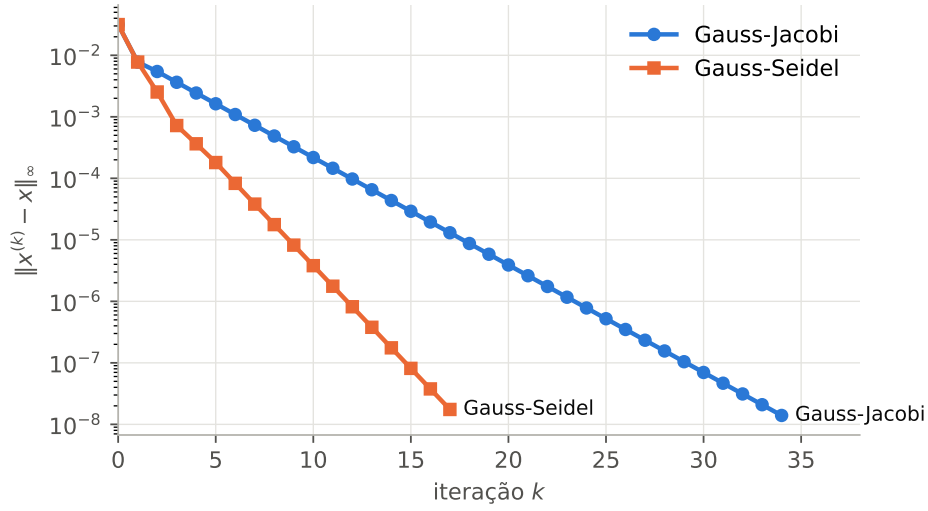


Figura 1: Exercício 2: erro $\|x^{(k)} - x\|_\infty$ de Jacobi e Seidel, com x dado pela eliminação de Gauss.

Tabela 3: Exercício 2: iterações de Gauss-Seidel (correntes em A).

k	$i_1^{(k)}$	$i_2^{(k)}$	$i_3^{(k)}$	$i_4^{(k)}$	$d^{(k)}$	$\ x^{(k)} - x\ _\infty$
0	0,0000000	0,0000000	0,0000000	0,0000000		$3,16 \times 10^{-2}$
1	-0,0285714	0,0261224	0,0071243	-0,0052016	1,00	$7,77 \times 10^{-3}$
2	-0,0233371	0,0306712	0,0064734	-0,0040107	$1,71 \times 10^{-1}$	$2,53 \times 10^{-3}$
3	-0,0215271	0,0309094	0,0069714	-0,0033360	$5,86 \times 10^{-2}$	$7,21 \times 10^{-4}$
4	-0,0211699	0,0312249	0,0073028	-0,0031180	$1,14 \times 10^{-2}$	$3,64 \times 10^{-4}$
5	-0,0209863	0,0314194	0,0074351	-0,0030199	$6,19 \times 10^{-3}$	$1,80 \times 10^{-4}$
6	-0,0208887	0,0315040	0,0074939	-0,0029719	$3,10 \times 10^{-3}$	$8,28 \times 10^{-5}$
7	-0,0208439	0,0315420	0,0075217	-0,0029496	$1,42 \times 10^{-3}$	$3,81 \times 10^{-5}$
8	-0,0208235	0,0315597	0,0075346	-0,0029393	$6,47 \times 10^{-4}$	$1,77 \times 10^{-5}$
9	-0,0208141	0,0315680	0,0075406	-0,0029345	$3,00 \times 10^{-4}$	$8,20 \times 10^{-6}$
10	-0,0208097	0,0315718	0,0075434	-0,0029323	$1,39 \times 10^{-4}$	$3,80 \times 10^{-6}$
11	-0,0208076	0,0315736	0,0075447	-0,0029313	$6,46 \times 10^{-5}$	$1,76 \times 10^{-6}$
12	-0,0208067	0,0315744	0,0075453	-0,0029308	$2,99 \times 10^{-5}$	$8,17 \times 10^{-7}$
13	-0,0208062	0,0315748	0,0075455	-0,0029306	$1,39 \times 10^{-5}$	$3,79 \times 10^{-7}$
14	-0,0208060	0,0315749	0,0075457	-0,0029305	$6,43 \times 10^{-6}$	$1,75 \times 10^{-7}$
15	-0,0208059	0,0315750	0,0075457	-0,0029304	$2,98 \times 10^{-6}$	$8,13 \times 10^{-8}$
16	-0,0208059	0,0315751	0,0075458	-0,0029304	$1,38 \times 10^{-6}$	$3,77 \times 10^{-8}$
17	-0,0208059	0,0315751	0,0075458	-0,0029304	$6,41 \times 10^{-7}$	$1,75 \times 10^{-8}$

Tabela 4: Exercício 2: iterações de Gauss-Jacobi (correntes em A).

k	$i_1^{(k)}$	$i_2^{(k)}$	$i_3^{(k)}$	$i_4^{(k)}$	$d^{(k)}$	$\ x^{(k)} - x\ _\infty$
0	0,0000000	0,0000000	0,0000000	0,0000000		$3,16 \times 10^{-2}$
1	-0,0285714	0,0342857	0,0000000	0,0000000	1,00	$7,77 \times 10^{-3}$
2	-0,0187755	0,0261224	0,0093506	-0,0077922	$3,75 \times 10^{-1}$	$5,45 \times 10^{-3}$
3	-0,0244474	0,0329287	0,0042908	-0,0017204	$2,07 \times 10^{-1}$	$3,64 \times 10^{-3}$
4	-0,0199005	0,0291396	0,0083550	-0,0051072	$1,56 \times 10^{-1}$	$2,44 \times 10^{-3}$
5	-0,0224346	0,0321806	0,0060900	-0,0023892	$9,45 \times 10^{-2}$	$1,63 \times 10^{-3}$
6	-0,0204009	0,0304858	0,0079077	-0,0039040	$6,67 \times 10^{-2}$	$1,09 \times 10^{-3}$
7	-0,0215343	0,0318459	0,0068947	-0,0026884	$4,27 \times 10^{-2}$	$7,28 \times 10^{-4}$
8	-0,0206248	0,0310879	0,0077077	-0,0033658	$2,93 \times 10^{-2}$	$4,87 \times 10^{-4}$
9	-0,0211317	0,0316962	0,0072546	-0,0028222	$1,92 \times 10^{-2}$	$3,26 \times 10^{-4}$
10	-0,0207249	0,0313572	0,0076182	-0,0031252	$1,30 \times 10^{-2}$	$2,18 \times 10^{-4}$
11	-0,0209516	0,0316293	0,0074155	-0,0028820	$8,60 \times 10^{-3}$	$1,46 \times 10^{-4}$
12	-0,0207696	0,0314776	0,0075782	-0,0030175	$5,78 \times 10^{-3}$	$9,75 \times 10^{-5}$
13	-0,0208710	0,0315993	0,0074875	-0,0029087	$3,85 \times 10^{-3}$	$6,52 \times 10^{-5}$
14	-0,0207897	0,0315315	0,0075603	-0,0029694	$2,58 \times 10^{-3}$	$4,36 \times 10^{-5}$
15	-0,0208350	0,0315859	0,0075197	-0,0029207	$1,72 \times 10^{-3}$	$2,91 \times 10^{-5}$
16	-0,0207986	0,0315556	0,0075523	-0,0029478	$1,15 \times 10^{-3}$	$1,95 \times 10^{-5}$
17	-0,0208189	0,0315799	0,0075341	-0,0029261	$7,71 \times 10^{-4}$	$1,30 \times 10^{-5}$
18	-0,0208026	0,0315664	0,0075487	-0,0029382	$5,16 \times 10^{-4}$	$8,72 \times 10^{-6}$
19	-0,0208117	0,0315773	0,0075406	-0,0029285	$3,45 \times 10^{-4}$	$5,83 \times 10^{-6}$
20	-0,0208044	0,0315712	0,0075471	-0,0029339	$2,31 \times 10^{-4}$	$3,90 \times 10^{-6}$
21	-0,0208085	0,0315761	0,0075435	-0,0029295	$1,54 \times 10^{-4}$	$2,61 \times 10^{-6}$
22	-0,0208052	0,0315733	0,0075464	-0,0029320	$1,03 \times 10^{-4}$	$1,74 \times 10^{-6}$
23	-0,0208070	0,0315755	0,0075447	-0,0029300	$6,90 \times 10^{-5}$	$1,17 \times 10^{-6}$
24	-0,0208056	0,0315743	0,0075460	-0,0029311	$4,61 \times 10^{-5}$	$7,80 \times 10^{-7}$
25	-0,0208064	0,0315753	0,0075453	-0,0029302	$3,08 \times 10^{-5}$	$5,22 \times 10^{-7}$
26	-0,0208057	0,0315747	0,0075459	-0,0029307	$2,06 \times 10^{-5}$	$3,49 \times 10^{-7}$
27	-0,0208061	0,0315752	0,0075456	-0,0029303	$1,38 \times 10^{-5}$	$2,33 \times 10^{-7}$
28	-0,0208058	0,0315749	0,0075458	-0,0029305	$9,23 \times 10^{-6}$	$1,56 \times 10^{-7}$
29	-0,0208060	0,0315751	0,0075457	-0,0029304	$6,17 \times 10^{-6}$	$1,04 \times 10^{-7}$
30	-0,0208058	0,0315750	0,0075458	-0,0029305	$4,13 \times 10^{-6}$	$6,98 \times 10^{-8}$
31	-0,0208059	0,0315751	0,0075457	-0,0029304	$2,76 \times 10^{-6}$	$4,67 \times 10^{-8}$
32	-0,0208058	0,0315751	0,0075458	-0,0029304	$1,85 \times 10^{-6}$	$3,12 \times 10^{-8}$
33	-0,0208059	0,0315751	0,0075458	-0,0029304	$1,23 \times 10^{-6}$	$2,09 \times 10^{-8}$
34	-0,0208059	0,0315751	0,0075458	-0,0029304	$8,25 \times 10^{-7}$	$1,40 \times 10^{-8}$

Tabela 5: Exercício 2: comparação entre o método direto e os iterativos (correntes em A).

Método	i_1	i_2	i_3	i_4	Iter.	$\ b - Ax\ _\infty$
Gauss c/ pivotamento	-0,0208059	0,0315751	0,0075458	-0,0029304		
Gauss-Jacobi	-0,0208059	0,0315751	0,0075458	-0,0029304	34	$8,6 \times 10^{-6}$
Gauss-Seidel	-0,0208059	0,0315751	0,0075458	-0,0029304	17	$3,3 \times 10^{-6}$

Resposta. As correntes são $i_1 = -20,81$ mA, $i_2 = 31,58$ mA, $i_3 = 7,55$ mA e $i_4 = -2,93$ mA. O sinal negativo de i_1 e i_4 só quer dizer que a corrente real circula no sentido contrário ao adotado na montagem do sistema. Para um sistema 4×4 o método direto é claramente o mais simples. Os iterativos fazem mais sentido para matrizes grandes e esparsas (slide 5), mas aqui serviram para ver na prática a diferença entre Jacobi e Seidel.

3.3 Exercício 3: parábola e elipse

Esboço. A parábola $y = x^2 + 1$ tem vértice em $(0, 1)$ e abre para cima. A elipse $x^2 + y^2/4 = 1$ tem semieixo 1 na direção x e 2 na direção y . Pelo desenho (Figura 2) as curvas se cruzam em dois pontos simétricos, um no primeiro quadrante e outro no segundo. Lendo do gráfico, tomamos como chutes iniciais $(0,7; 1,5)$ e $(-0,7; 1,5)$.

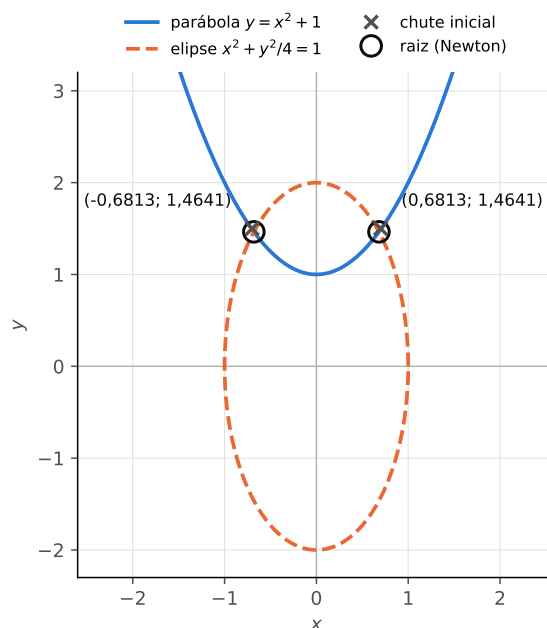


Figura 2: Exercício 3: esboço das curvas, chutes iniciais e raízes obtidas.

Formulação. Passando tudo para um lado, o problema fica

$$F\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} x^2 + 1 - y \\ x^2 + y^2/4 - 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad J_F(x, y) = \begin{bmatrix} 2x & -1 \\ 2x & y/2 \end{bmatrix},$$

com $\det J_F = xy + 2x = x(y + 2)$. A iteração de Newton é resolver

$$\begin{bmatrix} 2x_k & -1 \\ 2x_k & y_k/2 \end{bmatrix} \begin{bmatrix} s_x \\ s_y \end{bmatrix} = - \begin{bmatrix} x_k^2 + 1 - y_k \\ x_k^2 + y_k^2/4 - 1 \end{bmatrix}$$

e fazer $x^{(k+1)} = x^{(k)} + s^{(k)}$.

Implementação e resultados. Usamos a função `newton` da Seção 2.2 com $TOL = 10^{-10}$ no passo e $\varepsilon_1 = 10^{-12}$ no resíduo. Nos dois casos o método parou em 3 iterações

Tabela 6: Exercício 3: raiz do primeiro quadrante, chute (0,7; 1,5).

k	$x^{(k)}$	$y^{(k)}$	$\ F(x^{(k)})\ _\infty$	$\ s^{(k)}\ _\infty$	$\ x^{(k)} - x^*\ _\infty$	exp.	p_k
0	0,700000000000	1,500000000000	$5,2 \times 10^{-2}$	$3,6 \times 10^{-2}$	$3,6 \times 10^{-2}$	-2	
1	0,681632653061	1,464285714286	$6,6 \times 10^{-4}$	$3,8 \times 10^{-4}$	$3,8 \times 10^{-4}$	-4	
2	0,681250149606	1,464101620029	$1,5 \times 10^{-7}$	$1,1 \times 10^{-7}$	$1,1 \times 10^{-7}$	-7	1,79
3	0,681250038633	1,464101615138	$1,2 \times 10^{-14}$		$9,0 \times 10^{-15}$	-15	2,00

Tabela 7: Exercício 3: raiz do segundo quadrante, chute (-0,7; 1,5).

k	$x^{(k)}$	$y^{(k)}$	$\ F(x^{(k)})\ _\infty$	$\ s^{(k)}\ _\infty$	$\ x^{(k)} - x^*\ _\infty$	exp.	p_k
0	-0,700000000000	1,500000000000	$5,2 \times 10^{-2}$	$3,6 \times 10^{-2}$	$3,6 \times 10^{-2}$	-2	
1	-0,681632653061	1,464285714286	$6,6 \times 10^{-4}$	$3,8 \times 10^{-4}$	$3,8 \times 10^{-4}$	-4	
2	-0,681250149606	1,464101620029	$1,5 \times 10^{-7}$	$1,1 \times 10^{-7}$	$1,1 \times 10^{-7}$	-7	1,79
3	-0,681250038633	1,464101615138	$1,2 \times 10^{-14}$		$9,0 \times 10^{-15}$	-15	2,00

pelo critério do resíduo (Tabelas 6 e 7). Os expoentes do erro dobram a cada iteração e a última estimativa da ordem deu $p = 2,00$.

As raízes são (0,6812500386; 1,4641016151) e (-0,6812500386; 1,4641016151). As duas tabelas são iguais a menos do sinal de x , porque $F(-x, y) = F(x, y)$.

Neste exercício dá para conferir com a resposta exata. Substituindo $x^2 = y - 1$ na elipse chegamos em $y^2 + 4y - 8 = 0$, cuja raiz positiva é $y = 2\sqrt{3} - 2 = 1,464101615138$. Então $x = \pm\sqrt{y-1} = \pm 0,681250038633$. A diferença entre o Newton e esse valor exato foi de $9,2 \times 10^{-15}$.

Uma coisa que notamos testando outros chutes: o ponto (0; 1,5), que parece razoável no desenho por ficar entre as duas raízes, não funciona. Como $\det J_F = x(y+2)$, a Jacobiana é singular em todo o eixo y , e o `numpy` devolve o erro `Singular matrix` logo na primeira iteração.

3.4 Exercício 4

Formulação. Escrevemos o sistema como $F(x, y, z) = 0$ com

$$F(x, y, z) = \begin{bmatrix} 6x - 2y + e^z - 2 \\ \sin x - y + z \\ \sin x + 2y + 3z - 1 \end{bmatrix}, \quad J_F(x, y, z) = \begin{bmatrix} 6 & -2 & e^z \\ \cos x & -1 & 1 \\ \cos x & 2 & 3 \end{bmatrix}.$$

Como o enunciado pede a solução perto da origem, usamos $x^{(0)} = (0, 0, 0)$.

Como garantimos o erro. O enunciado pede erro menor que 10^{-5} em cada incógnita, que é o mesmo que pedir $\|x - x^*\|_\infty < 10^{-5}$ na norma do máximo. Usamos $\text{TOL} = 10^{-5}$ no passo e $\varepsilon_1 = 10^{-10}$ no resíduo. O argumento para confiar no passo vem da convergência quadrática (slide 17). Perto da raiz, $\|x^{(k+1)} - x^*\| \leq C \|x^{(k)} - x^*\|^2$, então $x^{(k+1)}$ está muito mais perto de x^* do que $x^{(k)}$. Assim $\|s^{(k)}\|_\infty = \|x^{(k+1)} - x^{(k)}\|_\infty$ é praticamente o erro de $x^{(k)}$. Quando $\|s^{(k)}\|_\infty < 10^{-5}$, o $x^{(k)}$ já tinha erro dessa ordem, e o valor devolvido pela função é o $x^{(k+1)}$, com erro da ordem de $\|s^{(k)}\|^2$.

Como o slide 24 lembra que isso não é uma prova, conferimos de duas formas. O resíduo final ficou em $1,4 \times 10^{-13}$. Também comparamos com a solução de referência

calculada no limite da precisão de máquina, e o erro de cada componente foi $4,9 \times 10^{-14}$ em x , $4,5 \times 10^{-14}$ em y e $6,8 \times 10^{-14}$ em z , bem abaixo de 10^{-5} .

Resultados. O método parou em 3 iterações pelo critério $\|s^{(k)}\|_\infty$ (Tabela 8). O último passo calculado teve tamanho $7,1 \times 10^{-7}$. O penúltimo iterado já tinha erro de $7,1 \times 10^{-7}$, praticamente igual a esse passo, e o último ficou com erro da ordem do quadrado do passo ($5,1 \times 10^{-13}$), o que bate com a tabela.

Tabela 8: Exercício 4: iterações de Newton a partir da origem.

k	$x^{(k)}$	$y^{(k)}$	$z^{(k)}$	$\ F(x^{(k)})\ _\infty$	$\ s^{(k)}\ _\infty$	$\ x^{(k)} - x^*\ _\infty$	exp.
0	0,0000000000	0,0000000000	0,0000000000	1,0	$3,0 \times 10^{-1}$	$3,0 \times 10^{-1}$	-1
1	0,2608695652	0,3043478261	0,0434782609	$2,9 \times 10^{-3}$	$2,4 \times 10^{-3}$	$2,4 \times 10^{-3}$	-3
2	0,2597517875	0,3027363339	0,0458954991	$3,1 \times 10^{-6}$	$7,1 \times 10^{-7}$	$7,1 \times 10^{-7}$	-7
3	0,2597510768	0,3027359948	0,0458960078	$1,4 \times 10^{-13}$		$6,8 \times 10^{-14}$	-14

A solução, com mais casas do que o pedido, é $x = 0,259751076760$, $y = 0,302735994802$ e $z = 0,045896007797$. Com seis casas, que já garantem o erro pedido, fica

$$(x, y, z) \approx (0,259751; 0,302736; 0,045896).$$

4 Parte 2: rede de adução por gravidade

As incógnitas são as três vazões e a carga no nó, $x = (Q_1, Q_2, Q_3, H)^\top$, e o sistema é o do enunciado:

$$F(x) = \begin{pmatrix} K_1 Q_1 |Q_1| + H - z_1 \\ K_2 Q_2 |Q_2| + H - z_2 \\ K_3 Q_3 |Q_3| + H - z_3 \\ Q_1 + Q_2 + Q_3 - q \end{pmatrix}, \quad K_i = \frac{8f_i L_i}{\pi^2 g D_i^5}.$$

Usamos o esqueleto do enunciado como base (a função F é a mesma) e completamos a Jacobiana. Em todos os itens o Newton usou $\text{TOL} = 10^{-10}$ no passo, $\varepsilon_1 = 10^{-10}$ no resíduo e no máximo 50 iterações.

4.1 Item (a): coeficientes e Jacobiana

Os valores de K_i estão na Tabela 9: $K_1 = 897,67$, $K_2 = 1.827,57$ e $K_3 = 3.045,96$, em s^2/m^5 .

Tabela 9: Item (a): dados das adutoras e coeficientes K_i .

Adutora i	f_i	L_i (m)	D_i (m)	K_i (s^2/m^5)
1	0,022	1.200	0,30	897,6745
2	0,024	900	0,25	1.827,5739
3	0,024	1.500	0,25	3.045,9565

Para a Jacobiana, a única derivada que dá trabalho é a de $KQ|Q|$. Para $Q > 0$ temos $KQ|Q| = KQ^2$, com derivada $2KQ = 2K|Q|$. Para $Q < 0$ temos $KQ|Q| = -KQ^2$, com derivada $-2KQ = 2K|Q|$. Em $Q = 0$ a derivada é zero, que também é $2K|Q|$. Então

$$\frac{d}{dQ}(KQ|Q|) = 2K|Q| \quad \text{para todo } Q.$$

Cada uma das três primeiras equações depende só da sua vazão e de H , e a última é a soma das vazões. Isso dá uma Jacobiana com formato de seta:

$$J(x) = \begin{pmatrix} 2K_1|Q_1| & 0 & 0 & 1 \\ 0 & 2K_2|Q_2| & 0 & 1 \\ 0 & 0 & 2K_3|Q_3| & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

Em Python ficou assim:

```
def J(x, K=K):
    # d/dQ (K Q|Q|) = 2 K |Q| -> diagonal; coluna de H e linha da
    # continuidade formam a "seta"
    Q = x[:3]
    M = np.zeros((4, 4))
    M[0, 0], M[1, 1], M[2, 2] = 2*K*np.abs(Q)
    M[:3, 3] = 1.0          # dF_i/dH = 1, i = 1, 2, 3
    M[3, :3] = 1.0         # dF_4/dQ_i = 1
    return M
```

Para ter certeza de que a derivada estava certa, comparamos J com a Jacobiana aproximada por diferenças progressivas do Newton discretizado (slide 20), com $h = 10^{-8}$. A maior diferença entre elas foi $3,1 \times 10^{-5}$ no ponto inicial do item (b) e $3,0 \times 10^{-5}$ na solução. No começo achamos estranho não dar algo perto de 10^{-8} , mas é o erro de truncamento da própria diferença progressiva (slide 21). Em $KQ|Q|$ esse erro é $(h/2) \cdot 2K = Kh$, e o maior deles é $K_3h = 3,0 \times 10^{-5}$, praticamente o valor que medimos. Então a Jacobiana analítica está certa.

4.2 Item (b): Newton a partir de $x^{(0)} = (0,10; 0,10; -0,05; 70)$

A Tabela 10 mostra as iterações. O método parou em 5 iterações pelo critério $\|F(x^{(k)})\|_\infty$. Na primeira iteração o erro aumentou, de 1,80 para 2,54, porque H pulou para 74,34 m. O chute não estava tão perto da raiz assim. A partir daí a convergência foi muito rápida. Outra coisa que reparamos: como a equação da continuidade é linear, depois do primeiro passo ela já fica satisfeita, e $|Q_1 + Q_2 + Q_3 - q|$ vale 0 a partir de $k = 1$.

Os expoentes do erro foram $0 \rightarrow 0 \rightarrow -2 \rightarrow -4 \rightarrow -9 \rightarrow -17$. Passada a primeira iteração, o número de casas corretas praticamente dobra a cada iteração, igual ao exemplo do slide 17, até chegar no limite da precisão de máquina na última linha. As estimativas da ordem deram 0,85, 2,63. A primeira ainda é da fase em que o método está se aproximando da raiz, e com tão poucas iterações na fase final a estimativa oscila em torno de 2. A Figura 3 mostra a mesma coisa em gráfico: a curva vai ficando cada vez mais inclinada, enquanto no Exercício 2 as curvas eram retas.

Então sim, os expoentes confirmam a convergência quadrática. Isso era o esperado pelo teorema do slide 19. A Jacobiana é Lipschitz, porque $|Q|$ é Lipschitz, e $J(x^*)$ é

Tabela 10: Item (b): iterações de Newton (vazões em m^3/s , H em metros).

k	$Q_1^{(k)}$	$Q_2^{(k)}$	$Q_3^{(k)}$	$H^{(k)}$	$\ F(x^{(k)})\ _\infty$	$\ x^{(k)} - x^*\ _\infty$	exp.	p_k
0	0,1000000000	0,1000000000	-0,0500000000	70,00000000	$2,1 \times 10^1$	1,8	0	
1	0,1929198605	0,0791618931	-0,0720817536	74,34089719	7,8	2,5	0	
2	0,1778228381	0,0851341004	-0,0629569385	71,81926708	$2,5 \times 10^{-1}$	$2,1 \times 10^{-2}$	-2	
3	0,1772481236	0,0849924886	-0,0622406123	71,79814896	$1,6 \times 10^{-3}$	$3,4 \times 10^{-4}$	-4	0,85
4	0,1772461190	0,0849912717	-0,0622373906	71,79849038	$3,2 \times 10^{-8}$	$7,0 \times 10^{-9}$	-9	2,63
5	0,1772461190	0,0849912716	-0,0622373906	71,79849039	$7,1 \times 10^{-15}$	$1,4 \times 10^{-17}$	-17	

não-singular, com $\det J(x^*) = -337.290,82$. Depois da primeira iteração o chute já estava perto o bastante.

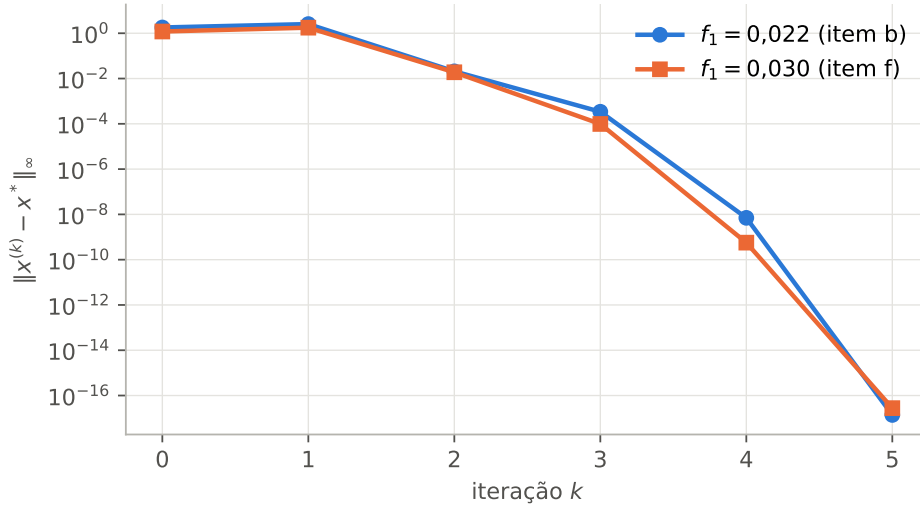


Figura 3: Parte 2: erro $\|x^{(k)} - x^*\|_\infty$ do Newton nos itens (b) e (f), a partir do mesmo $x^{(0)}$.

4.3 Item (c): o ponto $x^{(0)} = (0; 0; 0; 70)$

Com as três vazões nulas todos os termos $2K_i|Q_i|$ da diagonal zeram e

$$J(x^{(0)}) = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}, \quad \det J(x^{(0)}) = 0,0.$$

Olhando as linhas dá para ver o motivo: as três primeiras são iguais a $(0, 0, 0, 1)$. Linhas iguais são linearmente dependentes, então o determinante é zero e o posto da matriz é só 2. Fisicamente, com vazão zero a perda de carga $KQ|Q|$ tem derivada nula. Linearizando em torno desse ponto, as três equações de Bernoulli só enxergam H e não dizem nada sobre como mudar as vazões. As três primeiras equações de $Js = -F$ ficam $s_4 = z_i - H^{(0)}$, $i = 1, 2, 3$, ou seja, pedem três valores diferentes para a mesma correção de H . O sistema não tem solução e o Newton não consegue dar nem o primeiro passo.

Fórmula geral. Para ver se existem outros pontos com o mesmo problema, calculamos o determinante da matriz seta em geral. Chamando $d_i = 2K_i|Q_i| \geq 0$ e supondo primeiro

todos os $d_i \neq 0$, fazemos uma etapa de eliminação como no slide 9, com $L_4 \leftarrow L_4 - \frac{1}{d_1}L_1 - \frac{1}{d_2}L_2 - \frac{1}{d_3}L_3$. Isso zera a última linha, menos o último elemento, que vira $-\left(\frac{1}{d_1} + \frac{1}{d_2} + \frac{1}{d_3}\right)$. A matriz fica triangular superior e o determinante é o produto da diagonal:

$$\det J = d_1 d_2 d_3 \left(-\frac{1}{d_1} - \frac{1}{d_2} - \frac{1}{d_3} \right) = -(d_1 d_2 + d_1 d_3 + d_2 d_3).$$

O determinante é um polinômio nos d_i , então a fórmula continua valendo quando algum d_i é zero. Conferimos isso numericamente na Tabela 11.

Como os d_i nunca são negativos, a soma $d_1 d_2 + d_1 d_3 + d_2 d_3$ só zera quando pelo menos dois dos d_i são zero. Com um só zero, o produto dos outros dois continua positivo. E $d_i = 0$ exatamente quando $Q_i = 0$. Então a resposta é que sim, existem outros pontos com a mesma patologia. Qualquer ponto com pelo menos duas vazões nulas tem J singular, seja qual for H e seja qual for a terceira vazão. Por exemplo, em $(0, 0, Q_3, H)$ as duas primeiras linhas de J ficam iguais a $(0, 0, 0, 1)$. Já um ponto com só uma vazão nula não tem problema.

Tabela 11: Item (c): determinante pela fórmula e pelo `numpy` em alguns pontos iniciais, e o que o Newton fez em cada um.

Ponto $x^{(0)}$	$\det J$ (fórmula)	$\det J$ (<code>numpy</code>)	$\ s^{(0)}\ _\infty$	Newton
$(0; 0; 0; 70)$	0	0		<code>LinAlgError</code>
$(0; 0; 0,1; 70)$	0	0		<code>LinAlgError</code>
$(0,2; 0; 0; 50)$	0	0		<code>LinAlgError</code>
$(0,1; 0,1; 0; 70)$	$-6,56 \times 10^4$	$-6,56 \times 10^4$	$1,0 \times 10^1$	converge em 7 it.
$(10^{-3}; 10^{-3}; 10^{-3}; 70)$	$-3,98 \times 10^1$	$-3,98 \times 10^1$	$1,9 \times 10^1$	converge em 11 it.
$x^{(0)}$ do item (b)	$-2,32 \times 10^5$	$-2,32 \times 10^5$	4,3	converge em 5 it.

O que o código faz. Quando chamamos o `newton` a partir de $(0; 0; 0; 70)$, a função de resolver sistemas do `numpy` encontra um pivô nulo na fatoração LU e levanta o erro

```
numpy.linalg.LinAlgError: Singular matrix
```

Com o `inv` do código original aconteceria a mesma coisa. No script colocamos a chamada dentro de um `try/except`, que registra o erro na tabela e deixa o resto do script rodar. Testamos também um ponto perto do singular, $(10^{-3}; 10^{-3}; 10^{-3}; 70)$. Aí o método roda, mas o primeiro passo tem tamanho $1,9 \times 10^1$ e joga as vazões para até $6,1 \text{ m}^3/\text{s}$, muito longe da solução. Mesmo assim ele se recuperou e convergiu em 11 iterações, bem mais que as 5 do item (b).

4.4 Item (d): sentido do escoamento e velocidades

Na solução do item (b) encontramos $Q_1 = 0,177246$, $Q_2 = 0,084991$ e $Q_3 = -0,062237$ (em m^3/s), com $H = 71,7985$ m. Pela convenção do enunciado, $Q_i > 0$ quer dizer que o reservatório i alimenta o nó. Então $Q_3 < 0$ quer dizer que a água sai do nó e entra no reservatório 3. Isso acontece porque a carga no nó ficou acima do nível do reservatório 3 ($H - z_3 = 11,7985$ m). Os reservatórios 1 e 2 mandam juntos $0,262237 \text{ m}^3/\text{s}$ para o nó. Desse total, $0,200 \text{ m}^3/\text{s}$ vão para o bairro e $62,2 \text{ L/s}$ vão encher o reservatório 3.

Portanto a suspeita da companhia procede: na hora de pico o reservatório mais baixo está sendo enchido pelos outros dois e funciona como pulmão, e não como fonte.

As velocidades estão na Tabela 12. Os três valores, em módulo (2,51, 1,73 e 1,27 m/s), ficam dentro da faixa usual de projeto de 0,6 m/s a 3,0 m/s. A adutora 1 é a que está mais perto do limite de cima.

Tabela 12: Item (d): sentido do escoamento e velocidade média em cada adutora.

i	Q_i (m ³ /s)	Sentido	A_i (m ²)	v_i (m/s)	Na faixa?
1	0,177246	reservatório → nó	0,070686	2,508	sim
2	0,084991	reservatório → nó	0,049087	1,731	sim
3	-0,062237	nó → reservatório	0,049087	-1,268	sim

4.5 Item (e): demanda crítica

Queremos a demanda q^* em que $Q_3 = 0$. Seguindo o enunciado, fixamos $Q_3 = 0$ e passamos q para o lado das incógnitas, com $y = (Q_1, Q_2, H, q)$:

$$G(y) = \begin{pmatrix} K_1 Q_1 |Q_1| + H - z_1 \\ K_2 Q_2 |Q_2| + H - z_2 \\ H - z_3 \\ Q_1 + Q_2 - q \end{pmatrix}, \quad J_G(y) = \begin{pmatrix} 2K_1 |Q_1| & 0 & 1 & 0 \\ 0 & 2K_2 |Q_2| & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & -1 \end{pmatrix}.$$

A terceira equação ficou $H - z_3 = 0$, porque com $Q_3 = 0$ a perda de carga na adutora 3 some. Expandindo pela última coluna, $\det J_G = -(2K_1 |Q_1|)(2K_2 |Q_2|)$, que só zera se Q_1 ou Q_2 for nulo. Na solução vale -162.015,6.

Partimos da solução do item (b) com $q = 0,200$ e o Newton parou em 4 iterações pelo critério $\|F(x^{(k)})\|_\infty$ (Tabela 13). Como a terceira equação é linear, H já foi para 60 m na primeira iteração.

Tabela 13: Item (e): iterações de Newton para o sistema com q como incógnita.

k	$Q_1^{(k)}$	$Q_2^{(k)}$	$H^{(k)}$	$q^{(k)}$	$\ F(x^{(k)})\ _\infty$	$\ x^{(k)} - x^*\ _\infty$	exp.	p_k
0	0,1772461190	0,0849912716	71,79849039	0,2000000000	$1,2 \times 10^1$	$1,2 \times 10^1$	1	
1	0,2143227978	0,1229705911	60,00000000	0,3372933889	2,6	$9,2 \times 10^{-3}$	-3	
2	0,2111157659	0,1171056575	60,00000000	0,3282214233	$6,3 \times 10^{-2}$	$1,7 \times 10^{-4}$	-4	0,56
3	0,2110914071	0,1169587924	60,00000000	0,3280501995	$3,9 \times 10^{-5}$	$9,4 \times 10^{-8}$	-8	1,88
4	0,2110914057	0,1169587002	60,00000000	0,3280501059	$1,6 \times 10^{-11}$	$3,6 \times 10^{-14}$	-14	

O resultado foi $q^* = 0,328050$ m³/s (cerca de 328,1 L/s), com $Q_1 = 0,211091$ m³/s, $Q_2 = 0,116959$ m³/s e $H = 60,0000$ m. Para conferir, colocamos esse q^* no sistema original do item (b) e resolvemos de novo. Deu $Q_3 = 2,9 \times 10^{-14}$ m³/s, ou seja, zero no nível do arredondamento, e $H = 60,0000000000$ m.

A interpretação é a seguinte. Enquanto o consumo do bairro fica abaixo de q^* , a carga no nó fica acima de z_3 e o reservatório 3 recebe água. Acima de q^* ele passa a fornecer. Com a demanda de pico atual de 0,200 m³/s, abaixo de q^* , o reservatório 3 recebe, o que confere com o item (d).

4.6 Item (f): sensibilidade ao envelhecimento da adutora 1

Com $f_1 = 0,030$ o coeficiente da adutora 1 vira $K_1 = 1.224,10$, um aumento de 36,36 %. Refizemos o item (b), a partir do mesmo $x^{(0)}$, e o item (e). O Newton convergiu em 5 iterações, também de forma quadrática (Figura 3). A nova solução é $Q_1 = 0,159635$, $Q_2 = 0,094133$, $Q_3 = -0,053768$ (em m^3/s) e $H = 68,8058$ m, com $q^* = 0,297727$ m^3/s . A Tabela 14 compara com o caso original.

Tabela 14: Item (f): solução e demanda crítica antes e depois do aumento de f_1 . A variação é $100(\text{novo} - \text{antigo})/\text{antigo}$.

Grandeza	$f_1 = 0,022$	$f_1 = 0,030$	Variação (%)
Q_1 (m^3/s)	0,177246	0,159635	-9,94
Q_2 (m^3/s)	0,084991	0,094133	10,76
Q_3 (m^3/s)	-0,062237	-0,053768	-13,61
H (m)	71,7985	68,8058	-4,17
q^* (m^3/s)	0,328050	0,297727	-9,24

Pela variação relativa, a grandeza mais sensível entre as três perguntadas é a vazão Q_1 , com variação de -9,94 %. A demanda crítica veio logo atrás, com -9,24 %, e a carga no nó mudou bem menos, -4,17 %. Faz sentido Q_1 cair menos do que K_1 subiu: com a adutora 1 mais rugosa, H desce e os reservatórios 2 e 3 compensam parte da diferença (Q_2 aumentou e o reservatório 3 passou a receber só 53,8 L/s).

Discutindo o resultado, percebemos dois cuidados. O primeiro é que a porcentagem de H depende de onde está o zero das cotas. Se medirmos a carga a partir do nível do reservatório 3, $H - z_3$ vai de 11,7985 m para 8,8058 m, uma variação de -25,36 %. Por isso a comparação mais justa é entre Q_1 e q^* , que são vazões. O segundo é que em valor absoluto a demanda crítica cai mais: q^* perde 30,3 L/s e Q_1 perde 17,6 L/s. Para a operação isso importa, porque o consumo a partir do qual o reservatório 3 vira fonte cai para cerca de 297,7 L/s.

4.7 Conferência com `fsolve`

Por último, resolvemos os mesmos sistemas com o `scipy.optimize.fsolve`, só para conferir. Chamamos o `fsolve` sem passar a nossa Jacobiana. Ele aproxima a Jacobiana por conta própria, então a conferência também não depende da nossa derivada. A Tabela 15 mostra que em todos os casos o `fsolve` terminou com sucesso (`ier` = 1) e que a maior diferença para o nosso Newton foi de $3,6 \times 10^{-14}$.

Tabela 15: Conferência independente com `scipy.optimize.fsolve`.

Caso	<code>ier</code>	Avaliações de F	$\ x_{\text{fsolve}} - x_{\text{Newton}}\ _\infty$
item (b)	1	17	$1,4 \times 10^{-14}$
item (e)	1	15	$3,6 \times 10^{-14}$
item (f), solução	1	19	$2,8 \times 10^{-17}$
item (f), q^*	1	15	$4,2 \times 10^{-17}$

5 Código-fonte

O código completo está em <https://github.com/DaviGiuberti/TC2>. A pasta `src/` tem o arquivo `metodos.py`, com as funções reutilizáveis (normas, Gauss com pivotamento, Jacobi, Seidel, critérios de convergência, Newton e Newton discretizado), e um script para cada exercício: `parte1_ex1.py` a `parte1_ex4.py` e `parte2_aducao.py`. Cada script roda sozinho, por exemplo com `python src/parte2_aducao.py`, e grava as tabelas (`.csv` e `.tex`), as figuras e os valores usados neste relatório na pasta `resultados/`. O `README.md` explica como instalar as dependências e compilar o relatório.

6 Conclusão

Nos sistemas lineares, a eliminação de Gauss com pivotamento resolveu os dois problemas direto, e no Exercício 2 a dominância diagonal garantiu a convergência de Jacobi e Seidel. Seidel precisou de metade das iterações de Jacobi, como foi visto em aula.

Nos sistemas não-lineares, o Newton convergiu em poucas iterações sempre que o chute estava razoável, e os expoentes do erro dobraram como no slide 17. A parte que mais deu trabalho foi o chute inicial e os pontos em que a Jacobiana é singular. No Exercício 3 o eixo y inteiro é proibido, e na rede de adução qualquer ponto com duas vazões nulas também. Aprendemos a tratar esse erro no código em vez de deixar o script parar.

No problema da rede, os números confirmaram a suspeita da companhia: na hora de pico o reservatório 3 recebe água. Ele só passa a fornecer quando a demanda passa de q^* , e esse valor diminui com o envelhecimento da adutora 1.

Referências

- [1] SALLES NETO, L. L. *Resolução de Sistemas Lineares: métodos diretos e métodos iterativos*. Slides da disciplina Cálculo Numérico. São José dos Campos: ICT-UNIFESP, 2026.
- [2] SALLES NETO, L. L. *Normas Vetoriais*. Material complementar da aula de 28/08, Cálculo Numérico. São José dos Campos: ICT-UNIFESP, 2026.
- [3] SALLES NETO, L. L. *Métodos Numéricos para Sistemas Não-Lineares*. Slides da disciplina Cálculo Numérico. São José dos Campos: ICT-UNIFESP, 2026.
- [4] KHARAB, A.; GUENTHER, R. B. *An Introduction to Numerical Methods: A MATLAB Approach*. 4. ed. Boca Raton: CRC Press, 2018.
- [5] REAMAT. *Cálculo Numérico: um livro colaborativo, versão Python*. Porto Alegre: UFRGS. Seção 5.1.1, Método de Newton para sistemas. Disponível em: <https://www.ufrgs.br/reatmat/CalculoNumerico/livro-py/>.
- [6] NUMPY. *numpy.linalg.solve*. Documentação do NumPy. Disponível em: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html>.